FINAL PROJECT DOCUMENTATION

Project Title

Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets

Team ID : SWTID-2026-6274

Team Size : 4

Team Leader : DEBORA SHALINI T

Team Member : FAZIL AKRAM S

Team Member : MONISHA M S

Team Member : PRIYADHARSHINI S

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Project Overview

1.2 Purpose

2. IDEATION PHASE

2.1 Problem Statement

2.2 Empathy Map Canvas

2.3 Brainstorming

3. REQUIREMENT ANALYSIS

3.1 Customer Journey map

3.2 Solution Requirement

3.3 Data Flow Diagram

3.4 Technology Stack

4. PROJECT DESIGN

4.1 Problem Solution Fit

4.2 Proposed Solution

4.3 Solution Architecture

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

7. RESULTS

7.1 Output Screenshots

8. ADVANTAGES & DISADVANTAGES

9. CONCLUSION

10. FUTURE SCOPE

11. APPENDIX

Source Code(if any)

Dataset Link

GitHub & Project Demo Link

1. Introduction

In modern IT service management systems, maintaining data accuracy and consistency is essential for efficient operations. One of the key ways to achieve this is by ensuring that critical information is always captured during data entry. This can be accomplished by enforcing mandatory fields at the user interface level. Additionally, when developing and configuring such functionalities in platforms like ServiceNow, it is equally important to have a reliable method to move changes across different environments. This is where update sets play a vital role.

This project focuses on implementing UI policies to enforce mandatory fields and using update sets to migrate these configurations safely between instances.

1.1 Project Overview

This project involves two core components: enforcing mandatory fields using UI policies and migrating these configurations using update sets.

UI policies are used to dynamically control form behavior without requiring complex scripting. By configuring UI policies, specific fields can be made mandatory based on defined conditions, ensuring users provide essential data before submitting forms. This improves data quality and reduces errors in records.

The second part of the project focuses on update sets, which are used to track and transfer configuration changes from one environment (such as development) to another (such as testing or production). Update sets ensure that all customizations, including UI policies, are packaged and migrated systematically without loss of data or functionality.

Together, these components help streamline form validation and maintain consistency across multiple system environments.

1.2 Purpose

The purpose of this project is to enhance data integrity and ensure smooth configuration management within the system.

By enforcing mandatory fields through UI policies, the project ensures that users cannot submit incomplete or insufficient information. This leads to better decision-making, improved workflow efficiency, and reduced need for data correction.

At the same time, using update sets for migration provides a controlled and reliable way to move configurations across instances. It minimizes the risk of errors, maintains version control, and ensures that all implemented features are consistently deployed.

Overall, the project aims to demonstrate an effective approach to improving user input validation and managing system changes in a structured and efficient manner.

2. Ideation Phase

The ideation phase focuses on identifying the problem, understanding user needs, and generating effective solutions. In this project, the emphasis is on improving data accuracy through mandatory field enforcement and ensuring smooth migration of configurations using update sets.

2.1 Problem Statement

In many IT service management systems, users often submit forms with incomplete or missing critical information. This leads to data inconsistency, delays in processing requests, and increased manual effort to correct errors.

Additionally, organizations face challenges in migrating configurations such as UI policies from one environment to another. Without a proper mechanism, changes may be lost, duplicated, or inconsistently applied, leading to system inefficiencies.

Therefore, there is a need to implement a solution that enforces mandatory data entry at the user interface level and ensures reliable migration of configurations across environments.

2.2 Empathy Map Canvas

The empathy map helps in understanding the users interacting with the system (such as service desk agents, administrators, and end users).

1. Says

- “I want the form to be easy to fill.”
- “I don’t want to miss any required information.”
- “I need the system to guide me properly.”

2. Thinks

- Concerned about making mistakes while submitting forms
- Wants a faster and error-free submission process
- Expects system validations to help rather than hinder

3. Does

- Fills out forms quickly, sometimes skipping fields
- Submits requests without verifying all details
- Relies on system prompts or validations

4. Feels

- Frustrated when forms are rejected due to missing data
- Confused if requirements are unclear
- Satisfied when the system provides clear guidance

Insights Gained

- Users need clear, real-time validation
- Mandatory fields should be enforced intelligently
- The system should minimize confusion and rework

2.3 Brainstorming

During brainstorming, multiple solutions were considered to address the identified problem:

1. Manual Validation

- Checking forms manually after submission
- *Limitation:* Time-consuming and error-prone

2. Client Scripts

- Writing scripts to enforce mandatory fields
- *Limitation:* Requires coding and is harder to maintain

3. UI Policies (Selected Solution)

- Configuring conditions to make fields mandatory dynamically
- *Advantage:* No/low coding, easy to manage, efficient

4. Migration Approaches

- Manual reconfiguration in each environment
    - *Limitation:* Risk of inconsistency
- Using Update Sets (Selected Solution)
    - *Advantage:* Reliable, trackable, and efficient migration

Final Idea

- Use UI policies to enforce mandatory fields dynamically
- Use update sets to migrate all configurations safely across environments

3. Requirement Analysis

The requirement analysis phase focuses on understanding user needs, defining system requirements, and identifying the tools and processes required to implement the solution effectively. This ensures that the system meets user expectations and functions efficiently.

3.1 Customer Journey Map

The customer journey map illustrates the step-by-step interaction of the user with the system while filling and submitting a form.

Stages:

1. **Awareness**
    - User accesses the system to raise a request or incident
2. **Form Interaction**
    - User opens the form
    - Views available fields
3. **Data Entry**
    - User enters required information
    - UI policies dynamically enforce mandatory fields
4. **Validation**
    - System checks whether all mandatory fields are filled
    - Error messages are shown if any field is missing
5. **Submission**
    - User submits the form successfully after validation
6. **Post-Submission**
    - Data is stored in the database
    - Record is processed further
7. **System Update (Admin Perspective)**
    - Admin captures changes using update sets
    - Migrates configurations to other environments

Key Pain Points Addressed:

- Missing or incomplete data
- Lack of clear validation
- Errors during configuration migration

3.2 Solution Requirements

The solution requirements are divided into functional and non-functional requirements.

Functional Requirements

- The system must enforce mandatory fields using UI policies
- UI policies should be condition-based
- The system must validate form inputs before submission
- Error messages should be displayed for missing fields
- The system must support creation and management of update sets

- Update sets should allow migration between instances

Non-Functional Requirements

- **Usability:** Easy-to-use interface for users and administrators
- **Performance:** Fast validation without delays
- **Reliability:** Accurate enforcement of rules
- **Scalability:** Ability to handle multiple forms and users
- **Maintainability:** Easy configuration and updates

3.3 Data Flow Diagram (DFD)

Level 0 (Context Diagram):

- User interacts with the system
- System processes input and stores data
- Admin manages configurations and migrations

Flow:
User → Form Input → System Validation → Database

Admin → Configure UI Policy → Update Set → Target Instance

Level 1 (Detailed DFD):

1. User enters data into the form
2. UI policy checks conditions
3. System enforces mandatory fields
4. Validation process ensures completeness
5. Data is stored in the database
6. Admin creates update set
7. Update set is transferred and applied to another instance

3.4 Technology Stack

The technology stack used in this project includes:

- **Platform:** ServiceNow
- **Frontend:** Form-based User Interface (UI)
- **Backend:** ServiceNow database and server-side processing
- **Scripting (optional):** JavaScript (for client scripts if needed)
- **Configuration Tools:**

    - UI Policies
    - Update Sets

Tools & Features:

- Form Designer
- UI Policy Designer
- Update Set Management

4. Project Design

The project design phase focuses on defining how the identified problem will be effectively solved. It outlines the alignment between the problem and solution, describes the proposed approach, and presents the overall system architecture.

4.1 Problem–Solution Fit

The identified problem involves incomplete data entry in forms and difficulties in migrating configurations across environments. These issues lead to poor data quality, inefficiencies, and increased manual effort.

The proposed solution fits the problem effectively by addressing both concerns:

- **Problem:** Users submit forms with missing critical information
    - **Solution:** Enforce mandatory fields using UI policies
- **Problem:** Inconsistent and error-prone migration of configurations
    - **Solution:** Use update sets for structured and reliable migration

Why this is a good fit:

- UI policies provide real-time validation at the user interface level
- They eliminate the need for complex scripting
- Update sets ensure smooth and trackable migration of configurations
- The solution is scalable and aligned with industry practices

This alignment ensures that the solution directly resolves the core issues identified during analysis.

4.2 Proposed Solution

The proposed solution consists of two main components:

1. Enforcing Mandatory Fields Using UI Policies

- UI policies are configured to make specific fields mandatory based on conditions
- When a user interacts with a form, these policies dynamically enforce rules

- If required fields are not filled, the system prevents submission and displays error messages

Benefits:

- Improves data accuracy and completeness
- Provides better user guidance
- Reduces errors and rework

---

2. Migrating Changes Using Update Sets

- All configurations, including UI policies, are captured in update sets
- Update sets are used to move changes from one environment to another (e.g., Development → Testing → Production)
- Ensures consistency and avoids manual rework

Benefits:

- Reliable and efficient migration
- Version control and tracking of changes
- Reduces risk of configuration errors

4.3 Solution Architecture

The solution architecture illustrates how different components interact within the system.

Architecture Components:

1. User Interface Layer

- Users interact with forms
- UI policies are applied at this level

2. Application Logic Layer

- UI policies evaluate conditions
- Validation rules are enforced dynamically

3. Data Layer

- Stores validated records in the database

4. Configuration & Migration Layer

- Update sets capture configuration changes
- Changes are migrated across instances

Architecture Flow:

User → Form Interface → UI Policy Execution
→ Field Validation → Data Submission → Database Storage

Admin → Create UI Policy → Capture in Update Set
→ Transfer Update Set → Apply in Target Instance

Key Design Features:

- Condition-based validation
- Minimal scripting approach
- Modular and scalable design
- Secure and consistent migration process

5. Project Planning & Scheduling

5.1 Project Planning

Project planning is a crucial phase that defines how the project will be executed, monitored, and completed within the given time frame. It involves identifying tasks, allocating resources, and setting milestones to ensure the successful implementation of the solution.

In this project, planning was carried out systematically to ensure smooth execution of all activities related to enforcing mandatory fields using UI policies and migrating changes using update sets.

Project Phases and Activities

The project was divided into the following phases:

1. **Requirement Analysis**
    - Understanding the problem statement
    - Identifying user needs and system requirements
2. **Design Phase**
    - Creating system architecture
    - Designing flowcharts and diagrams
3. **Development Phase**
    - Configuring UI policies
    - Implementing validation rules
4. **Testing Phase**
    - Verifying mandatory field enforcement
    - Testing update set migration

5. **Deployment Phase**
    - Moving configurations to target environment
    - Ensuring system stability
6. **Documentation Phase**
    - Preparing project report
    - Recording implementation details

Resource Planning

- **Human Resources:**
    - Team members (developers, analysts, documentation support)
- **Technical Resources:**
    - ServiceNow platform
    - Internet access
    - System for testing and deployment

Time Planning

The project was scheduled over a defined timeline with specific milestones:

- Week 1: Problem identification and requirement analysis
- Week 2: Design and planning
- Week 3: Development and configuration
- Week 4: Testing and migration using update sets
- Week 5: Documentation and final review

Risk Management

Potential risks and mitigation strategies:

- **Risk:** Misconfiguration of UI policies
    - *Mitigation:* Testing in development environment
- **Risk:** Errors during update set migration
    - *Mitigation:* Validate update sets before deployment
- **Risk:** Incomplete requirements
    - *Mitigation:* Regular team discussions and reviews

Outcome of Planning

Effective project planning ensured:

- Smooth workflow and coordination
- Timely completion of tasks
- Proper implementation of features
- Reduced errors and rework

---

6. Functional and Performance Testing

Testing is an essential phase in the project to ensure that the system works as expected and meets both functional and performance requirements. It helps in identifying errors, verifying system behavior, and ensuring reliability before deployment.

6.1 Performance Testing

Performance testing focuses on evaluating the system's responsiveness, speed, and stability under different conditions. In this project, performance testing ensures that enforcing mandatory fields using UI policies does not negatively impact system performance and that update set migration operates efficiently.

Objectives of Performance Testing

- To verify that UI policies execute quickly without delays
- To ensure form submission is smooth even with multiple mandatory fields
- To check system stability during multiple user interactions
- To evaluate the efficiency of update set migration

Performance Test Scenarios

1. **Form Load Time**
    - Measure the time taken to load forms with UI policies applied
    - Ensure no noticeable delay for users
2. **Field Validation Speed**
    - Check how quickly mandatory fields are enforced when conditions are met
    - Ensure real-time validation
3. **Concurrent Users**
    - Simulate multiple users accessing and submitting forms simultaneously
    - Verify system stability under load

4. **Update Set Migration Performance**
    - Measure the time taken to transfer and apply update sets
    - Ensure no data loss or configuration issues

Performance Metrics

- **Response Time:** Time taken for UI policies to apply changes
- **Throughput:** Number of form submissions handled within a given time
- **Latency:** Delay between user action and system response
- **Error Rate:** Number of failures during validation or migration

---

Testing Results (Sample)

- UI policies executed instantly without noticeable delay
- Form validation worked in real time
- System handled multiple users efficiently
- Update set migration completed successfully without errors

Conclusion of Performance Testing

The system performed efficiently under all tested conditions. UI policies did not introduce performance overhead, and update sets ensured smooth and reliable migration. This confirms that the solution is both scalable and suitable for real-world implementation.

7. Results

The results section demonstrates the successful implementation of the project. It highlights how the system enforces mandatory fields using UI policies and ensures seamless migration of configurations using update sets. Screenshots of the actual output help validate the functionality.

7.1 Output Screenshots

1. Form with Mandatory Fields Enforced

- Shows a form where UI policies have dynamically made certain fields mandatory based on conditions.
- Users cannot submit the form until all required fields are filled.

---

2. Successful Form Submission

- Demonstrates that the form submission is only allowed after filling all mandatory fields.

- Ensures data integrity and completeness.

---

3. Update Set Creation

- Shows capturing of UI policy configurations in an update set for migration.

---

4. Update Set Migration

- Demonstrates moving the update set from the development environment to another instance (testing/production).
- Ensures configurations are applied correctly without loss.

---

5. Validation of Migrated Configuration

- Confirms that UI policies and mandatory field enforcement work in the target environment after migration.

8. Advantages & Disadvantages

This section highlights the key benefits and limitations of implementing mandatory fields using UI policies and migrating changes with update sets. Understanding both helps in evaluating the effectiveness of the solution.

8.1 Advantages

1. **Improved Data Accuracy**
    - Mandatory fields ensure users provide all necessary information before submitting a form.
2. **Real-Time Validation**
    - UI policies dynamically enforce rules, providing immediate feedback to users.
3. **Reduced Errors and Rework**
    - Ensures completeness of data, reducing the need for manual corrections.
4. **Efficient Migration**
    - Update sets allow seamless transfer of configurations across instances.
5. **Minimal Coding Required**
    - UI policies provide a low-code/no-code solution for field validation.
6. **Scalable and Flexible**
    - Can be applied to multiple forms and environments with minimal changes.
7. **Consistency Across Environments**

- Ensures that configurations remain consistent in development, testing, and production.

8.2 Disadvantages

1. **Dependent on Correct Configuration**
   - Improperly configured UI policies can block valid form submissions.
2. **Limited to Platform Features**
   - Functionality relies on the capabilities of the ServiceNow platform (or similar platforms).
3. **Migration Risks**
   - Incorrectly captured update sets may lead to incomplete or failed migrations.
4. **Maintenance Overhead**
   - Multiple UI policies and update sets require regular monitoring and updates.
5. **Performance Impact**
   - Excessive or complex UI policies may slightly affect form load times.

---

9. Conclusion

The project **"Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets"** successfully demonstrates how to improve data accuracy and consistency in IT service management systems. By implementing UI policies, mandatory fields are dynamically enforced, reducing errors and ensuring complete data submission.

Using update sets for migration ensures that all configurations, including UI policies, can be transferred reliably between environments without manual rework or loss of data. The project highlights a low-code, efficient, and scalable approach to form validation and system configuration management.

Overall, the solution meets the project objectives, improves operational efficiency, and provides a structured method for handling both user input validation and environment migration.

---

10. Future Scope

The project can be extended and enhanced in several ways:

1. **Advanced Validation Rules**
   - Incorporate complex conditional logic to make fields mandatory based on multiple criteria.
2. **Integration with Workflows**
   - Combine mandatory field enforcement with automated workflows to trigger notifications, approvals, or alerts.

3. **Reporting & Analytics**
    - Generate reports to analyze data quality and identify recurring submission errors.
4. **Cross-Platform Compatibility**
    - Extend similar functionality to other ITSM platforms or web-based applications.
5. **Automation of Migration**
    - Automate update set creation, testing, and deployment for faster and error-free migration.
6. **User Training & Self-Learning**
    - Add contextual help or tooltips for users to understand mandatory field requirements.

These improvements can further enhance system efficiency, reduce human errors, and provide a better user experience.

---

11. Appendix

GitHub & Project Demo Link

- GitHub Repository: https://github.com/deborashalini123-prog/SERVICE-NOW-
- Project Demo: https://drive.google.com/file/d/1bKOj_HuvEPW6QVkiv1T5wHP3f-r_9zVs/view?usp=drivesdk